

Homework 2

Ravi Kini

November 3, 2025

Exercise 1

Let A be a $m \times n$ strictly column diagonally dominant matrix. Then for all k :

$$|A_{kk}| > \sum_{i \neq k} |A_{ik}|$$

For Gaussian elimination with partial pivoting, row k of matrix A is interchanged only when for some $i > k$:

$$|A_{k,k}| \geq |A_{i,k}|$$

Note that for strictly column diagonally dominant matrices, no row interchange occurs since $|A_{kk}| > \sum_{i \neq k} |A_{ik}| > |A_{jk}|$ for all j, k . It then follows that after k steps of Gaussian elimination, if the submatrix excluding the first k columns/rows is strictly column diagonally dominant, no row interchange occurs for the $k + 1$ th step. Let $A^{(k)}$ denote the matrix after the k th step of Gaussian elimination is performed (i.e., $A^{(0)} = A$). We show by induction that after the k th step, the submatrix excluding the first k columns/rows is strictly column diagonally dominant; since A is already strictly column diagonally dominant, the base case of $k = 0$ holds. Suppose after the $k - 1$ th step, the hypothesis holds. We can then select the k th row as the pivot row, and $A_{k,k}^{(k-1)}$ as the pivot element. Then, for all $i > k$, using the triangle inequality:

$$\begin{aligned} \sum_{j > k, j \neq i} |A_{j,i}^{(k)}| &= \sum_{j > k, j \neq i} \left| A_{j,i}^{(k-1)} - A_{j,k}^{(k-1)} \frac{A_{k,i}^{(k-1)}}{A_{k,k}^{(k-1)}} \right| \\ &\leq \sum_{j > k, j \neq i} |A_{j,i}^{(k-1)}| + \sum_{j > k, j \neq i} |A_{j,k}^{(k-1)}| \frac{|A_{k,i}^{(k-1)}|}{|A_{k,k}^{(k-1)}|} \\ &\leq \sum_{j > k, j \neq i} |A_{j,i}^{(k-1)}| + |A_{k,i}^{(k-1)}| \frac{\sum_{j > k, j \neq i} |A_{j,k}^{(k-1)}|}{|A_{k,k}^{(k-1)}|} \\ &\leq \sum_{j \geq k, j \neq i} |A_{j,i}^{(k-1)}| - |A_{j,k}^{(k-1)}| + |A_{k,i}^{(k-1)}| \frac{\sum_{j > k} |A_{j,k}^{(k-1)}| - |A_{i,k}^{(k-1)}|}{|A_{k,k}^{(k-1)}|} \end{aligned}$$

Using the strict column diagonal dominance of the submatrix of $A^{(k-1)}$ that excludes the first $k-1$ columns/rows:

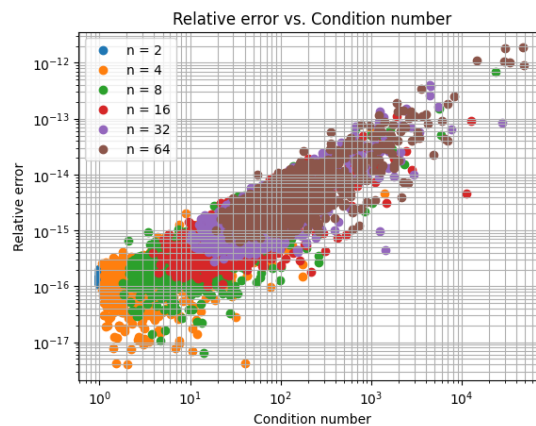
$$\begin{aligned}
\sum_{j>k, j \neq i} |A_{j,i}^{(k)}| &\leq |A_{i,i}^{(k-1)}| - |A_{k,i}^{(k-1)}| + |A_{k,i}^{(k-1)}| \frac{|A_{k,k}^{(k-1)}| - |A_{i,k}^{(k-1)}|}{|A_{k,k}^{(k-1)}|} \\
&\leq |A_{i,i}^{(k-1)}| - |A_{k,i}^{(k-1)}| + |A_{k,i}^{(k-1)}| \left(1 - \frac{|A_{i,k}^{(k-1)}|}{|A_{k,k}^{(k-1)}|}\right) \\
&\leq |A_{i,i}^{(k-1)}| - |A_{k,i}^{(k-1)}| \frac{|A_{i,k}^{(k-1)}|}{|A_{k,k}^{(k-1)}|} \\
&\leq \left| A_{i,i}^{(k-1)} - A_{k,i}^{(k-1)} \frac{A_{i,k}^{(k-1)}}{A_{k,k}^{(k-1)}} \right| \\
&\leq |A_{i,i}^{(k)}|
\end{aligned}$$

The submatrix of $A^{(k)}$ that excludes the first k columns/rows is then strictly column diagonally dominant. By induction, this holds for every step of Gaussian elimination, and so row interchange does not occur when performing Gaussian elimination on a strictly column diagonally dominant matrix.

Exercise 2

Part (a)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4
5 N = 10**3
6
7 min_exp, max_exp = 1, 6
8
9 rel_error_arr = np.empty((max_exp - min_exp + 1, N))
10 kappa_arr = np.empty((max_exp - min_exp + 1, N))
11
12 for j in range(max_exp - min_exp + 1):
13     for i in range(N):
14         n = 2**j
15         A = np.random.rand(n, n) * 2- 1
16         z = np.random.rand(n, 1) * 2- 1
17         b = np.matmul(A, z)
18         x = np.linalg.solve(A, b)
19         kappa = np.linalg.cond(A)
20         rel_error = np.linalg.norm(x - z) / np.linalg.norm(z)
21         rel_error_arr[j][i] = rel_error
22         kappa_arr[j][i] = kappa
23
24 for j in range(max_exp - min_exp + 1):
25     plt.scatter(kappa_arr[j], rel_error_arr[j], label=f'n = {2 ** (min_exp + j)}')
26 plt.xscale('log')
27 plt.yscale('log')
28 plt.xlabel('Condition number')
29 plt.ylabel('Relative error')
30 plt.title('Relative error vs. Condition number')
31 plt.grid(True, which='both', ls='-')
32 plt.legend()
33 plt.show()
34
35 def fit_function(x, a, k):
36     return a * x ** k
37 for j in range(max_exp - min_exp + 1):
38     popt, pcov = sp.optimize.curve_fit(fit_function, kappa_arr[j], rel_error_arr[j])
39     print(popt)
```



There is a linear relationship between the logs of the condition number and of the relative error with a slope of approximately 1, which indicates that $\frac{\|x-z\|}{\|z\|} \propto \kappa(A)$.

Part (b)

The scaling is not dependent on the matrix size.

Part (c)

The intercept on the log-log plot is approximately machine epsilon, which means that the constant of proportionality between the condition number and the relative error is approximately machine epsilon, as expected.

Exercise 3

$$\rho = \frac{\max_{i,j} |U_{i,j}|}{\max_{i,j} |A_{i,j}|}$$

Part (a)

For a $n \times n$ matrix A , let $A^{(k)}$ denote the matrix after the k th step of Gaussian elimination is performed (i.e., $A^{(0)} = A$). We first show that $\max_{i,j} |A_{i,j}^{(k)}| \leq 2^k \max_{i,j} |A_{i,j}|$. The base case holds trivially since $\max_{i,j} |A_{i,j}^{(0)}| = \max_{i,j} |A_{i,j}|$. Suppose the bound holds for the first $k-1$ steps. At step k , we choose the pivot $A_{k,k}^{(k-1)}$ such that:

$$|A_{k,k}^{(k-1)}| \geq |A_{i,k}^{(k-1)}|$$

for all $i > k$. Consequently:

$$\frac{|A_{i,k}^{(k-1)}|}{|A_{k,k}^{(k-1)}|} \leq 1$$

For all $i \neq k$ and all j :

$$\begin{aligned} |A_{i,j}^{(k)}| &= \left| A_{i,j}^{(k-1)} - \frac{A_{i,k}^{(k-1)}}{A_{k,k}^{(k-1)}} A_{k,j}^{(k-1)} \right| \\ &\leq |A_{i,j}^{(k-1)}| + \left| \frac{A_{i,k}^{(k-1)}}{A_{k,k}^{(k-1)}} A_{k,j}^{(k-1)} \right| \\ &\leq |A_{i,j}^{(k-1)}| + |A_{k,j}^{(k-1)}| \\ &\leq 2 \max_{p,q} |A_{p,q}^{(k-1)}| \\ &\leq 2 \cdot 2^{k-1} \max_{p,q} |A_{p,q}| \\ &\leq 2^k \max_{i,j} |A_{i,j}| \end{aligned}$$

For all j :

$$\begin{aligned} |A_{k,j}^{(k)}| &= |A_{k,j}^{(k-1)}| \leq 2^{k-1} \max_{i,j} |A_{i,j}| \\ &\leq 2^k \max_{i,j} |A_{i,j}| \end{aligned}$$

Therefore $\max_{i,j} |A_{i,j}^{(k)}| \leq 2^k \max_{i,j} |A_{i,j}|$, and by induction, this holds for all k . Note that in the LU decomposition of A , if $A = LU$, $U = A^{(n-1)}$. Then:

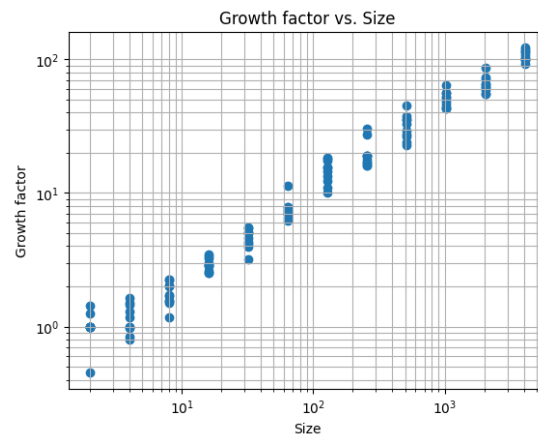
$$\begin{aligned} \max_{i,j} |U_{i,j}| &= \max_{i,j} |A_{i,j}^{(n-1)}| \leq 2^{n-1} \max_{i,j} |A_{i,j}| \\ \rho &= \frac{\max_{i,j} |U_{i,j}|}{\max_{i,j} |A_{i,j}|} \leq 2^{n-1} \end{aligned}$$

Part (b)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4 import scipy as sp
5
6 N = 10
7 min_exp, max_exp = 1, 12
8
9 n_arr = np.empty((max_exp - min_exp + 1) * N)
10 rho_arr = np.empty((max_exp - min_exp + 1) * N)
11
12 for j in range(min_exp, max_exp + 1):
13     for i in range(N):
14         n = 2**j
15         A = np.random.rand(n, n) * 2 - 1
16         P, L, U = sp.linalg.lu(A)
17         rho = np.max(U) / np.max(A)
18         n_arr[(j - 1) * N + i] = n
19         rho_arr[(j - 1) * N + i] = rho
20
21 plt.scatter(n_arr, rho_arr)
22 plt.xscale('log')
23 plt.yscale('log')
24 plt.xlabel('Size')
25 plt.ylabel('Growth factor')
26 plt.title('Growth factor vs. Size')
27 plt.grid(True, which='both', ls='-')
28 plt.show()
29
30 def fit_function(x, a, k):
31     return a * x ** k
32 popt, pcov = sp.optimize.curve_fit(fit_function, n_arr, rho_arr)
33 print(popt)

```



There is a linear relationship between the logs of the size and of the growth factor with a slope of approximately 0.6 and intercept of approximately 0.75, which indicates that the $\rho \approx 0.75n^{0.6}$.

Part (c)

For a matrix of size $10^6 \times 10^6$, I expect a growth factor of about $0.75(10^6)^{0.6} \approx 2985.8$. This is far lower than the bound from part (a), which indicates that for most matrices, Gaussian elimination with partial pivoting is numerically stable.

Exercise 4

Part (a)

Let A be a $m \times n$ matrix. Let i^* such that $\|A\|_\infty = \sum_{j=1}^n |A_{i^*,j}|$. Define the vector x such that:

$$x_j = \frac{\text{sgn}(A_{i^*,j})}{\sqrt{n}}$$

Evidently, $\|x\|_2 = n \cdot \left(\frac{1}{\sqrt{n}}\right)^2 = 1$. Then:

$$\begin{aligned} (Ax)_i &= \frac{1}{\sqrt{n}} \sum_{j=1}^n |A_{i,j}| \\ (Ax)_{i^*} &= \frac{\|A\|_\infty}{\sqrt{n}} \end{aligned}$$

We see that $(Ax)_{i^*} \geq (Ax)_i$ for all i . Then:

$$\|A\|_2 \geq \|Ax\|_2 \geq (Ax)_{i^*} \geq \frac{\|A\|_\infty}{\sqrt{n}}$$

For an arbitrary x such that $\|x\|_2 = 1$:

$$\|Ax\|_2^2 = \sum_{i=1}^m \left| \sum_{j=1}^n A_{i,j} x_j \right|^2$$

By the Cauchy-Schwarz inequality:

$$\begin{aligned} \left| \sum_{j=1}^n A_{i,j} x_j \right| &\leq \sum_{j=1}^n |A_{i,j}| |x_j| \\ &\leq \sum_{j=1}^n |A_{i,j}| \\ &\leq \|A\|_\infty \end{aligned}$$

Then:

$$\begin{aligned} \|Ax\|_2^2 &\leq \sum_{i=1}^m \|A\|_\infty^2 \\ &\leq m \|A\|_\infty^2 \end{aligned}$$

Taking the supremum over all such x :

$$\begin{aligned}\|A\|_2^2 &\leq m\|A\|_\infty^2 \\ \|A\|_2 &\leq \sqrt{m}\|A\|_\infty\end{aligned}$$

Consequently:

$$\frac{1}{\sqrt{n}}\|A\|_\infty \leq \|A\|_2 \leq \sqrt{m}\|A\|_\infty$$

Part (b)

For the lower bound, let A be the $m \times n$ matrix such that:

$$A_{i,j} = \begin{cases} 1 & i = 1 \\ 0 & \text{otherwise} \end{cases}$$

Clearly, $\|A\|_\infty = n$. We see that $A^T A$ is the matrix with all entries as 1, which has the vector $(1, \dots, 1)^T$ as its only eigenvector and n as the corresponding eigenvalue; then $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sqrt{n}$.

For the upper bound, let A be the $m \times n$ matrix such that:

$$A_{i,j} = \begin{cases} 1 & j = 1 \\ 0 & \text{otherwise} \end{cases}$$

Clearly, $\|A\|_\infty = 1$. We see that $A^T A$ is the matrix with zero in all entries except $(A^T A)_{1,1} = m$, which has the vector $(1, 0, \dots, 1)^T$ as its only eigenvector and m as the corresponding eigenvalue; then $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sqrt{m}$.

Part (c)

Let A be a $n \times n$ square matrix. From the above inequalities, we can bound the condition number of A and A^{-1} by:

$$\begin{aligned}\frac{1}{\sqrt{n}}\|A\|_\infty &\leq \|A\|_2 \leq \sqrt{n}\|A\|_\infty \\ \frac{1}{\sqrt{n}}\|A^{-1}\|_\infty &\leq \|A^{-1}\|_2 \leq \sqrt{n}\|A^{-1}\|_\infty\end{aligned}$$

Then:

$$\begin{aligned}\frac{1}{n}\|A\|_\infty\|A^{-1}\|_\infty &\leq \|A\|_2\|A^{-1}\|_2 \leq n\|A\|_\infty\|A^{-1}\|_\infty \\ \frac{1}{n}\kappa_\infty(A) &\leq \kappa_2(A) \leq n\kappa_\infty(A)\end{aligned}$$

These bounds are likely not tight since they would require both A and A^{-1} to take on extreme values simultaneously; the matrices used to show the tightness of the bounds in part (b), in particular, are singular and therefore not invertible.

Exercise 5

Part (a)

$$\begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix}$$

The following algorithm solves the above tridiagonal system of equations:

```

1 x=zeros(n,1);
2 w=zeros(n,1);
3 d = b(1);
4 x(1) = f(1)/d;
5 for i = 2:n
6     w(i-1) = c(i-1) / d;
7     d = b(i) - a(i) * w(i-1);
8     x(i) = (f(i) - a(i) * x(i-1))/d;
9 end
10 for i = n-1:-1:1
11     x(i) = x(i) - w(i) * x(i + 1);
12 end

```

The LU decomposition of the tridiagonal matrix has the form:

$$\begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ & & & a_n & b_n \end{bmatrix} = \begin{bmatrix} 1 & & & & \\ \alpha_2 & 1 & & & \\ 0 & \ddots & \ddots & & \\ 0 & 0 & \ddots & \ddots & \\ 0 & 0 & 0 & \alpha_n & 1 \end{bmatrix} \begin{bmatrix} \beta_1 & c_1 & 0 & 0 & 0 \\ \beta_2 & c_2 & 0 & 0 & 0 \\ & \ddots & \ddots & \ddots & 0 \\ & & \ddots & \ddots & c_{n-1} \\ & & & \ddots & \beta_n \end{bmatrix}$$

where α_j and β_j can be defined recursively, with:

$$\begin{aligned} \beta_1 &= b_1 \\ \alpha_j &= \frac{a_j}{\beta_{j-1}} \\ \beta_j &= b_j - \alpha_j c_{j-1} \end{aligned}$$

Part (b)

If the system of equations is modified as follows:

$$\begin{bmatrix} b_1 & c_1 & & & a_1 \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ c_n & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} f_1 \\ \vdots \\ f_n \end{bmatrix}$$

the above algorithm is no longer applicable. However, note that the modified matrix can be decomposed as follows:

$$\begin{bmatrix} b_1 & c_1 & & & a_1 \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ c_n & & & a_n & b_n \end{bmatrix} = \begin{bmatrix} b_1 - 1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ & & & a_n & b_n - a_1 c_n \end{bmatrix} + \begin{bmatrix} 1 \\ \vdots \\ \vdots \\ \vdots \\ c_n \end{bmatrix} \begin{bmatrix} 1 \\ \vdots \\ \vdots \\ \vdots \\ a_1 \end{bmatrix}^T$$

$$A = B + uv^T$$

We can compute the solutions to the following using the above algorithm:

$$By = f$$

$$Bq = u$$

We then argue that $x = y - \frac{qv^T y}{1+v^T q}$. To see why this is true:

$$\begin{aligned} Ax &= (B + uv^T) \left(y - \frac{qv^T y}{1+v^T q} \right) \\ &= By - \frac{Bqv^T y}{1+v^T q} + uv^T y - \frac{uv^T qv^T y}{1+v^T q} \\ &= f - \frac{uv^T y}{1+v^T q} + uv^T y - (v^T q) \frac{uv^T y}{1+v^T q} \\ &= f + uv^T y - (1+v^T q) \frac{uv^T y}{1+v^T q} \\ &= f + uv^T y - uv^T y \\ &= f \end{aligned}$$

Modifying the above algorithm:

```

1 % modify tridiagonal array and construct u and v
2 b(1) = b(1) - 1;
3 b(n) = b(n) - a(1) * c(n);
4 u = zeros(n,1);
5 u(1) = 1;
6 u(n) = a(1);
7 v = zeros(n,1);
8 v(1) = 1;
9 v(n) = c(n);
10
11 % solve By = f
12 y = zeros(n,1);
13 w = zeros(n,1);
14 d = b(1);
15 y(1) = f(1)/d;
16 for i = 2:n
17     w(i-1) = c(i-1) / d;
18     d = b(i) - a(i) * w(i-1);

```

```

19     y(i) = (f(i) - a(i) * y(i-1))/d;
20 end
21 for i = n-1:-1:1
22     y(i) = y(i) - w(i) * y(i + 1);
23 end
24
25 % solve Bq = u
26 q = zeros(n,1);
27 w = zeros(n,1);
28 d = b(1);
29 q(1) = u(1)/d;
30 for i = 2:n
31     w(i-1) = c(i-1) / d;
32     d = b(i) - a(i) * w(i-1);
33     q(i) = (u(i) - a(i) * q(i-1))/d;
34 end
35 for i = n-1:-1:1
36     q(i) = q(i) - w(i) * q(i + 1);
37 end
38
39 % construct x
40 x = y - q .* (v' * y) ./ (1 + (v' * q))

```